



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 6

NOMBRE COMPLETO: Ricardo Emmanuel Galicia Tequianes

N° de Cuenta: 118001740

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 4 de octubre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA 6 TEXTURIZADO

La práctica compara el texturizado de una malla construida en C++ con la importación de modelos OBJ y sus materiales. La escena integra dos holocrones y una recreación del avión verde de la referencia. Se utiliza el shader de texturas del material Practica6271 y se conserva la carga de texturas repetidas para el piso.

1 Ejecución de los ejercicios

Ejercicio 1 Holocrón creado por código

CrearHolocron define seis caras independientes. Cada cara tiene cuatro vértices con posición, coordenadas UV y normal, y se dibuja con dos triángulos. El total es de 24 vértices y 12 triángulos. Separar las caras permite asignar una imagen completa a cada una sin compartir coordenadas UV en las aristas.

Las seis imágenes finales son las proporcionadas para esta entrega. Se guardan como cara_1.png a cara_6.png; el archivo ORIGEN_CARAS.txt relaciona cada nombre con su original. Se conserva el contenido de las imágenes, incluidas las marcas visibles. La versión WEBP se convierte a PNG para que stb_image pueda cargarla.



Figura 1. Captura real de OpenGL. Holocrón generado por código, a la izquierda, con las nuevas imágenes.

```
// practica6.cpp - carga de las seis imagenes
for(int i=1;i<=6;++i) {
    auto t=new Texture(("Textures/cara_"+std::to_string(i)+".png").c_str());
    if(!t->LoadTextureA(GL_CLAMP_TO_EDGE)) return 2;
    caras.push_back(t);
}
// Una malla por cara: cuatro vertices y seis indices.
```

Bloque 1. Extracto del código de carga; representación legible del archivo fuente.

Ejercicio 2 Holocrón modelado e importado

El holocrón se modeló mediante las herramientas de Blender: cubo, modo de edición y bisel de 0.12 unidades. La geometría no se generó mediante un script de Python. Se prepararon UV por proyección cúbica y materiales de imagen. La asignación final distribuye las seis imágenes entre las caras principales; los biseles tienen un material independiente.

El archivo OBJ contiene posiciones v, coordenadas vt, normales vn y caras f. La biblioteca MTL asocia los materiales con las imágenes mediante map_Kd. Assimp carga estos datos y Model.cpp crea las mallas de OpenGL, conserva su índice de material y activa la textura correspondiente antes de dibujar.

Para sustituir el atlas inicial por las seis imágenes nuevas se ajustaron las referencias de material y UV del OBJ. El archivo editable final se conserva en Blender junto con las texturas y el OBJ usado por el programa. Este ajuste de datos se distingue del modelado de la geometría realizado en la interfaz.



Figura 2. Captura real del holocrón importado. El borde biselado permite distinguirlo del cubo del ejercicio 1.

```
// practica6.cpp - importacion de recursos
Model imported,plane;
imported.LoadModel("Models/holocron_modelado.obj");
plane.LoadModel("Models/avion_verde.obj");
// Model.cpp vincula material, textura y malla con Assimp.
```

Bloque 2. Carga de los dos modelos externos desde el programa principal.

Resultado: el modelo importado conserva las seis caras texturizadas y se dibuja mediante la misma cámara, proyección y shader que la malla del primer ejercicio.

Ejercicio 3 Avión con rasgos del personaje

El avión se construyó en Blender a partir de primitivas escaladas: fuselaje, alas, estabilizador, timón, cabina y hélice. Después se exportó a OBJ. Los ajustes posteriores de UV y materiales separan tres regiones: ojos en el vidrio frontal, nariz y sonrisa en el frente del fuselaje y motivos de llamas en las alas.

Se tomó como referencia el avión verde adjunto. Las texturas de ojos, cara y llamas son dibujos vectoriales propios, guardados también como SVG, y se rasterizaron a PNG para OpenGL. El resultado es una recreación simplificada: conserva el verde y los motivos del personaje, pero no pretende ser el modelo tridimensional original ni una copia idéntica de sus proporciones.

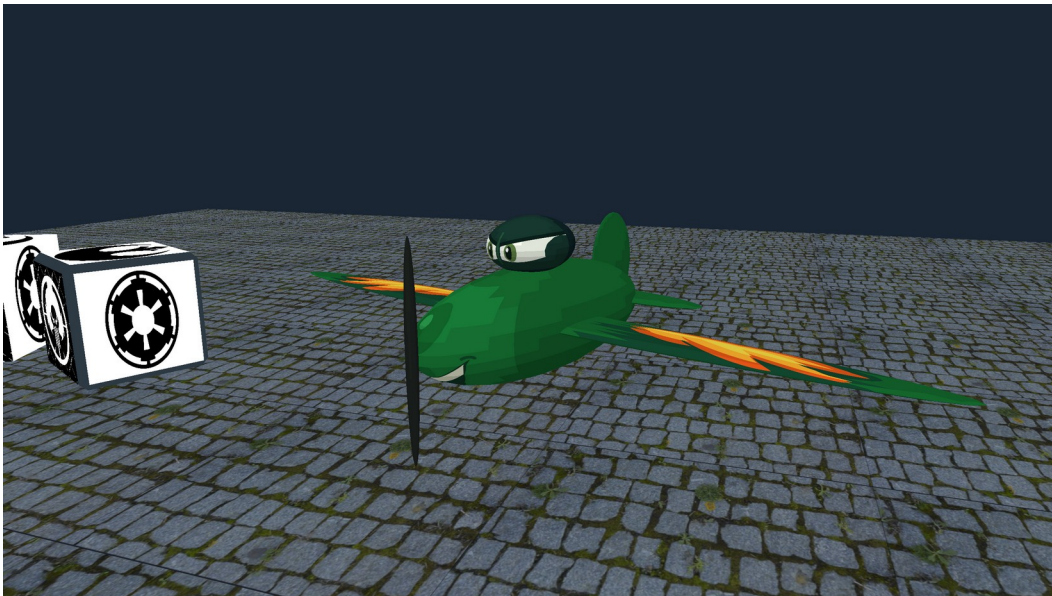


Figura 3. Captura real del avión importado con ojos, nariz, sonrisa y motivos de las alas.

Las variaciones de verde se almacenan en los materiales y pequeñas texturas de color. No se añadió un modelo de iluminación al shader del profesor. La hélice es geometría estática; los holocrones pueden girar con la tecla R.

```
// shader_texture.frag - original del material del curso
void main()
{
    color = vColor*texture(theTexture, TexCoord);
}
```

Bloque 3. Operación del shader de fragmentos del curso: combina color y muestra de textura.

La imagen original avion_referencia.png se incluye como referencia visual; las texturas utilizadas por la cara del avión son ojos.png, cara.png y llamas.png.

2 Problemas y soluciones

Carga de recursos al abrir desde Finder. Las rutas relativas dependían del directorio de ejecución. Se agregó la localización de Resources a partir de la ruta del ejecutable en macOS; así, modelos, imágenes y shaders se encuentran al abrir la aplicación.

Validación prematura del shader. La validación antes de configurar el estado de dibujo podía fallar y evitar la obtención de las ubicaciones uniform. Se conserva la comprobación de compilación y enlazado, y se eliminó esa salida prematura. No se cambió el contenido de `shader_texture.vert` ni `shader_texture.frag`.

Selección de envoltura. `LoadTextureA` recibe `wrapMode` y conserva `GL_REPEAT` como valor predeterminado. Las caras se cargan explícitamente con `GL_CLAMP_TO_EDGE`; el piso utiliza `GL_REPEAT` con coordenadas que repiten la imagen. La envoltura determina qué ocurre fuera del intervalo UV; `GL_LINEAR` es el filtro de muestreo y se configura por separado.

Alineación de memoria. La carga solicita `STBI_rgb_alpha` y sube los datos como `GL_RGBA`. Cada fila ocupa `ancho × 4` bytes, por lo que es compatible con la alineación predeterminada de cuatro bytes. No se usa `glPixelStorei(GL_UNPACK_ALIGNMENT, 1)`, porque esta ruta de carga no lo necesita.

Asociación de materiales. Se corrigió la condición de acceso al vector de texturas para comprobar primero que el índice está dentro del rango. También se normalizaron separadores de ruta y se conserva una textura de respaldo cuando un material no tiene imagen.

Legibilidad del avión. La referencia de 348×348 píxeles producía rasgos borrosos al usar regiones pequeñas como textura. Se prepararon dibujos separados para los ojos, la boca y las llamas. La cabina y la hélice se colocaron de forma que los rasgos fueran visibles.

Entradas de teclado. Las teclas de acción se conservan hasta que el ciclo de dibujo las procesa, evitando perder pulsaciones breves. Las teclas de movimiento siguen respondiendo mientras se mantienen presionadas.

Verificación en macOS

El proyecto compiló con CMake y AppleClang para ARM64. La ejecución reportó OpenGL 4.1 Metal y GLSL 4.10. Se capturaron las vistas 1, 2, 3 y 4; `glGetError` devolvió 0 en las capturas. El registro y las imágenes se incluyen en la carpeta Evidencias. Estas comprobaciones corresponden al Mac de trabajo; no se afirma una validación en Windows o Linux.

3 Conclusiones

Complejidad y explicación

El primer ejercicio permite controlar directamente los vértices y las UV, pero exige definir correctamente cada cara. El segundo añade la relación entre OBJ, MTL y archivos de imagen. El avión requiere una separación más cuidadosa de regiones porque una misma proyección no coloca correctamente ojos, boca y motivos de las alas.

Comentarios generales

La distinción más útil para esta práctica es entre geometría, UV, material y shader. Una imagen correcta puede verse mal si las coordenadas la recortan o si el material apunta a otra ruta. Conviene verificar cada etapa con un modelo simple antes de revisar toda la escena. También resulta útil explicar por separado la envoltura, el filtrado y la alineación de memoria.

Conclusión

Los ejercicios muestran dos formas de llevar una textura a OpenGL: definir sus coordenadas en C++ o importarlas junto con una malla. Ambas terminan usando la misma operación de muestreo del shader del curso. Conservar las imágenes, el archivo editable y los materiales permite revisar el resultado y cambiar una cara sin reconstruir el programa completo.

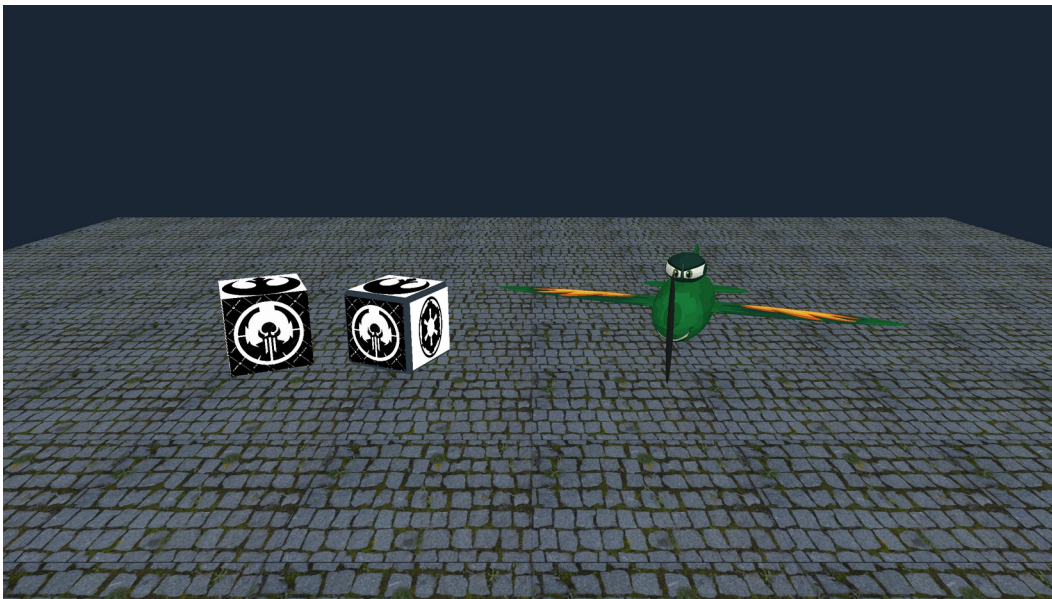


Figura 4. Escena integrada: holocrón por código, holocrón importado y avión.

Controles: 1, 2 y 3 seleccionan cada ejercicio; 4 restaura la vista general; W, A, S y D desplazan la cámara; botón derecho y movimiento del ratón cambian la orientación; R activa o detiene la rotación de los holocrones; F12 guarda una captura; Esc cierra la ventana.

Bibliografía

Roque RG, J. (2026). Practica6271 y consigna de la práctica 6: Texturizado [Material de clase]. Laboratorio de Computación Gráfica e Interacción Humano Computadora, Facultad de Ingeniería, UNAM.

Blender Foundation. (s. f.). Wavefront OBJ. Blender Manual.
https://docs.blender.org/manual/en/5.0/files/import_export/obj.html

The Khronos Group. (s. f.). glTexParameter. OpenGL Wiki.
<https://wikis.khronos.org/opengl/GLAPI/glTexParameter>

The Khronos Group. (s. f.). OpenGL reference pages. <https://registry.khronos.org/OpenGL-Refpages/>

Procedencia de los recursos

Las seis caras del holocrón y la referencia del avión fueron proporcionadas para esta práctica. No se atribuye la autoría de esos diseños al estudiante. Los shaders de texturas proceden de Practica6271; las clases de cámara, ventana y shader se integraron a partir del material local del curso. Las texturas vectoriales del avión se crearon para esta implementación.

Correspondencia de las caras, en el orden de las imágenes recibidas: frente (+Z), fondo (-Z), derecha (+X), izquierda (-X), superior (+Y) e inferior (-Y). Los nombres originales completos se conservan en Textures/ORIGEN_CARAS.txt.

Archivos de la entrega

Practica6 incluye el código C++, CMakeLists.txt, Models, Textures, shaders y Evidencias. Practica6Texturizado.app es la aplicación compilada para este Mac. LEEME.md contiene las instrucciones de apertura y compilación. Los archivos del curso en Descargas se conservan sin modificaciones.